

TESTING

Il testing è una delle fasi più **cruciali** nello sviluppo del software.

Il testing è il processo che mira a rilevare evidenze di difetti (defects) nel software.

Non serve a “*dimostrare che il software funziona*”, ma piuttosto a **scoprire dove non funziona**.

Il testing **comprende gli sforzi** per identificare i difetti, non per risolverli. Significa che l'obiettivo del tester è **rivelare** i malfunzionamenti, **non correggerli**.

Il testing **non include** il lavoro di **debugging** (cioè la ricerca della causa del difetto) né la **correzione** del codice. Queste attività spettano agli **sviluppatori**, non ai tester.

Quindi, Testing = individuare l'esistenza di un errore.

DEFINIZIONI DI BASE.

Un **bug** (o defect) è la **causa interna** del malfunzionamento. È un errore nel codice sorgente, nella logica o nella progettazione, che **porta a un failure** durante l'esecuzione.

Il **bug** è la radice del problema, non la sua manifestazione.

Il **failure** è ciò che si vede: l'effetto del problema. Significa comportamento inatteso del sistema, ossia una manifestazione esterna di un errore.

Sostanzialmente: Error (sviluppatore) genera un defect (nel codice) che causa un failure (durante l'esecuzione).

Best practices del software testing.

- 1.** Inizia i test il prima possibile, già nelle fasi iniziali del ciclo di vita del software. Non bisogna aspettare che tutto il codice sia finito.
- 2.** Il testing deve essere **continuo e iterativo**, non un evento isolato alla fine dello sviluppo. È una pratica chiave nelle metodologie **agili**. Testare spesso riduce il rischio di introdurre regressioni e mantiene il software stabile nel tempo.
- 3.** Non basta testare tanto: bisogna testare **ciò che conta davvero**, con la **tecnica più adeguata**.

PAROLE CHIAVE IMPORTANTI

Un *test case* è una specifica situazione in cui si testa il software fornendo determinati **input** e confrontando gli **output** ottenuti con gli **output** attesi.

Esempio.

Input: somma(2,3), *Expected result:* 5.

Test Case Execution: È la fase di esecuzione del test, in cui si forniscono gli input al sistema e si confrontano i risultati reali con quelli attesi.

Se actual(reale) result == expected result → test **passa**.

Se actual(reale) result ≠ expected result → test **fallisce**.

Test Suites: Un insieme organizzato logicamente di **test cases**. Il raggruppamento in test suites è **manuale**. Il raggruppamento è a livello di **classi**, non a livello di metodo.

Potrei avere 100 casi di test totali di cui, ad esempio, 30 per la parte grafica, 30 per la parte logica e 40 per la zona relativa alla scrittura/interfacciamento con il database.

Se io modificassi il database, vorrei poter nuovamente eseguire tutti i casi di test che riguardano la base di dati, piuttosto che tutti e 100: $40/100 = 0,4 = 40\%$ di copertura.

E quindi, nella pratica, si crea una test suites relativa ai casi di test del database.

Lo stesso test case può essere collegato a più test suites contemporaneamente.

Nota: *Per 10 righe di codice potrei avere bisogno di 1000 righe di test.*

Per una semplice divisione, che mi può occupare 2 righe di codice, io potrei avere 10 casi di test.

Vuoi testare che la divisione per 0 funziona?

Vuoi testare che la divisione per un numero negativo funziona?

Vuoi testare che il resto funziona?

Vuoi testare che per un numero N superiore a un certo valore X la divisione continua a funzionare?

Morale: all'aumentare della complessità del sistema bisogna imparare a capire quale test valga la pena eseguire.

Testare tutti i casi è impossibile. Ma proprio a **livello pratico**.

Adequacy: Misura quanto la suite di test è **adeguata/testata o sufficiente** a verificare il sistema.

In pratica: “Abbiamo testato *abbastanza*?”

E da qui nasce il concetto di **coverage**.

COVERAGE

Coverage è una *percentuale* che misura **quanto del codice è stato effettivamente coperto dai test**.

Il coverage indica quale percentuale del programma è stata realmente verificata tramite i casi di test.

Serve a valutare se i test coprono in modo adeguato il comportamento del software.

$$\text{Coverage} = \frac{\text{elementi coperti dai test}}{\text{elementi totali}} \times 100$$

Ad esempio: Se ho 100 istruzioni nel programma e i miei test ne eseguono 80, ho un Coverage = 80%.

Tuttavia, alta coverage $\neq$ assenza di bug.

Non è detto che una coverage del 100% mi dia 0 errori.

Il coverage misura solo quanto codice è stato eseguito dai test, non se il comportamento è corretto. Significa che potresti eseguire ogni riga di codice, ogni condizione, ogni ramo, ma se i test non **verificano** i risultati, gli errori possono comunque passare inosservati.

Black-box testing

È una tecnica in cui il tester non conosce la struttura interna del codice. Il sistema viene trattato come una “*scatola nera*”: si forniscono input e si osservano gli output per verificare se **rispettano le specifiche**. Può essere **preparato in anticipo**, anche **prima della programmazione**, se le specifiche sono già definite.

Supponiamo di testare una funzione che deve accettare numeri da **1 a 10**.

- Input testati: 0, 1, 5, 10, 11
- Expected results: errore per 0 e 11, validi per 1-10: non guardo come è scritto il codice, mi baso solo sulla regola data.

White-box testing

Il white box testing si basa sulla struttura interna del codice: viene scritto dopo che l'applicazione è scritta.

Per questo è detto anche: **Structural testing**.

Il tester **conosce la struttura interna del codice** e scrive test che mirano a **coprire** istruzioni, rami, percorsi e condizioni.

Conoscendo il codice, posso assicurarmi che **ogni parte venga testata**, inclusi i rami o i percorsi che potrebbero sfuggire nel black-box testing.

Se il programmatore **ha dimenticato di implementare** una parte prevista nella specifica, il white box testing **non la scoprirà**, perché analizza solo il codice esistente: banalmente non puoi testare ciò che non è stato scritto.

Esempio:

```
int divide(int a, int b) {  
    if (b != 0)  
        return a / b;  
    else  
        return 0;  
}
```

Black box test:

- Specifica: *“la funzione deve gestire divisione per zero”*;
- Test: `divide(10, 0)` e mi aspetto 0;

White box test:

- Guardo il codice e vedo due percorsi:
 1. `b != 0`;
 2. `b == 0`;

I **code smells** sono un concetto fondamentale che si collega direttamente alla **qualità del codice** e, di conseguenza, anche al **testing** e alla **manutenibilità** del software.



Un **code smell** (“odore di codice”) è come un **cattivo odore** in casa: non è detto che ci sia un incendio (cioè un bug), ma **qualcosa non va e potrebbe peggiorare col tempo**. Durante il **white-box testing** (strutturale), l’analisi del codice può rivelare **code smells**.

Il **refactoring** è l’insieme di tecniche per migliorare la qualità del codice senza cambiare il suo comportamento. Si usa per eliminare i code smells.

TEST UNITARIO

Immagina di costruire una **macchina**. Prima di testare l’intera auto, provi **ogni pezzo separatamente**: il motore funziona da solo? i freni rispondono? il volante gira bene?

Questo è il **test unitario**!



Un **test unitario** è un test che verifica il comportamento corretto di una **unità di codice isolata**, solitamente una funzione, metodo o classe, per assicurarsi che produca i risultati attesi dati specifici input.

L'API che supporta il test unitario è **JUnit** (www.junit.org).

The programmer-friendly testing framework for Java
and the JVM

User Guide

Javadoc

Code & Issues

Q & A

Sponsor

JUnit è un framework open-source per il testing automatizzato in Java, usato principalmente per il **unit testing** ma anche per **integration testing**.

Funzionalità principali (JUnit features)

1. Assertions for testing expected results.

Le assertions servono a verificare automaticamente i risultati attesi.

Esempio: `assertEquals(5, somma(2,3));`

Generico: `assertEquals(expected, actual);`

Significa: “mi aspetto che la somma di 2 e 3 sia 5”.

JUnit confronterà l’output con il valore atteso e segnalerà errore se **diverso**.

È la parte più importante: consente di **automatizzare la verifica**, senza dover controllare a mano.

2. Test suites for organizing and running tests.

Una **test suite** è un insieme di test che puoi eseguire **tutti insieme** in un solo comando. Esempio: puoi creare una suite che include tutti i test del progetto:

```
@RunWith(Suite.class)
```

```
@Suite.SuiteClasses({
```

```
    CalcolatriceTest.class,
```

```
    LoginTest.class,
```

```
    PagamentoTest.class
```

```
});
```

```
public class AllTests {}
```

Quindi, test suites puntatori a classi di test che a loro volta sono puntatori a metodi di test.

3. Test features for sharing common test data.

JUnit permette di riutilizzare dati e oggetti comuni tra più test.

```
@BeforeEach
```

```
void setUp() {  
  
    db = new DatabaseConnection();  
  
}
```

Questo metodo viene eseguito **prima di ogni test**, così tutti i test possono usare la stessa connessione o configurazione.

FLESSIBILITÀ

JUnit è un framework molto flessibile: può essere usato per testare **diversi livelli di granularità del codice**, non solo singoli metodi.

Intero oggetto: Puoi testare il **comportamento complessivo di una classe**, controllando che tutte le sue funzionalità lavorino correttamente insieme.

Hai una classe Calcolatrice con metodi `somma()`, `sottrai()`, `moltiplica()`. Puoi scrivere un test che verifica il funzionamento globale della classe, assicurandoti che tutte le operazioni siano coerenti.

Una parte di un oggetto: Puoi testare **solo un metodo specifico**, o un piccolo gruppo di metodi che collaborano. Questo è il caso tipico del **test unitario** vero e proprio. Testi solo il metodo `somma()` della classe Calcolatrice, senza interessarti agli altri.

L'interazione tra più oggetti: JUnit permette anche di testare **come più oggetti interagiscono tra loro**, ad esempio, una classe controller che usa un DAO o un service. Questo tipo di test è detto **integration test**. `PagamentoController` chiama `PagamentoDAO`, scrivi un test per verificare che il controller gestisca correttamente la risposta del DAO.

JUnit fornisce **overload** di `assertEquals()` per diversi tipi di dati:

<code>assertEquals(boolean expected, boolean actual)</code>	Verifica che due valori booleani siano uguali
<code>assertEquals(byte expected, byte actual)</code>	Verifica che due byte siano uguali
<code>assertEquals(char expected, char actual)</code>	Verifica che due caratteri siano uguali
<code>assertEquals(int expected, int actual)</code>	Verifica che due numeri interi siano uguali
<code>assertEquals(double expected, double actual, double delta)</code>	Verifica che due numeri <i>double</i> siano uguali entro una certa tolleranza (delta)
<code>assertEquals(float expected, float actual, float delta)</code>	Come sopra, ma per <i>float</i>

Attenzione: l'ordine conta, sempre il risultato atteso e dopo quello reale.

Da questo momento in poi, il vostro progetto da consegnare dovrà includere un nuovo package, denominato **test**.

Quando aggiungi test con **JUnit**, il progetto viene organizzato in **due rami principali di codice**:

`src/main/ java/*codice applicativo*`
`src/test/java/*codice test*`

Quando aggiungerai i test con **JUnit**, dovrai creare un **package test dedicato**, separato dal codice principale. È lo **standard di fatto** nei progetti Java Maven e nelle consegne accademiche.

Ad esempio, il mio `LoginControllerTest` può stare tranquillamente in:

`src/test/java/Controller/LoginControllerTest.java`

Quindi, suddividendo le cartelle nel modo appena descritto, distinguiamo il codice di test da quello applicativo.


```

package test;

import logic.BasicCalculations;
import org.junit.Test;
import static org.junit.Assert.*;

public class TestDivision {

    @Test
    public void testDivision() {

        BasicCalculations b = new BasicCalculations();
        double output = b.perfectDivision(11, 10);
        assertEquals((double) 2, output, 1);

    }

}

```

Questa è una classe di test JUnit (versione 4, non 5) ed è strutturata nel modo corretto per verificare una funzione chiamata `perfectDivision()` della classe `BasicCalculations`.

LA TOLLERANZA

Quando fai calcoli con numeri decimali (ad esempio $0.1 + 0.2$), il risultato può essere leggermente impreciso per via del modo in cui il computer rappresenta i numeri in memoria.

Per questo, JUnit ti fa definire una “tolleranza” (delta), cioè un piccolo margine di errore accettabile.

Esempio:

```
assertEquals(double expected, double actual, double delta)
```

expected: il valore atteso (teorico)

actual: il valore calcolato dal programma

delta: la differenza massima accettabile tra i due

Il test **passa** se

$|\text{expected} - \text{actual}| \leq \text{delta}$

Se la differenza è **maggiore** del delta, il test **fallisce**.

Esempio:

@Test

```
void testCalcoloPiGreco() {  
    double risultato = 3.1415926;  
    assertEquals(3.14, risultato, 0.01); // delta = 0.01  
}
```

Il test **passa** perché $|3.1415926 - 3.14| = 0.0015926$, che è **minore** di 0.01.

Pratiche d'uso migliori.

1. I test devono essere **separati** dal codice di produzione.

2. Se un metodo può avere più comportamenti o casi limite, scrivi **più test** per coprirli tutti.

```
@Test void testDivisioneValida() { ... }
@Test void testDivisionePerZero() { ... }
@Test void testDivisioneNegativi() { ... }
```

3. Ogni metodo di test deve contenere una sola asserzione. Questo semplifica la diagnosi degli errori: se il test fallisce, sai esattamente quale condizione non è soddisfatta. Se devi testare più casi dello stesso metodo, puoi usare una variabile comune ma creare più metodi di test.

4. Convenzione di naming: se la classe è Calcolatrice.java, la classe di test sarà TestCalcolatrice.java.

```
// File: src/test/java/it/mio/progetto/TestCalcolatrice.java
package it.mio.progetto;
```

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class TestCalcolatrice {
```

```
    @Test
    void testSomma() {
        Calcolatrice calc = new Calcolatrice();
        assertEquals(5, calc.somma(2, 3));
    }
```

```
    @Test
    void testDivisioneNormale() {
        Calcolatrice calc = new Calcolatrice();
        assertEquals(2, calc.dividi(10, 5));
    }
```

```
    @Test
    void testDivisionePerZero() {
```

```
        Calcolatrice calc = new Calcolatrice();
        assertThrows(IllegalArgumentException.class, () ->
calc.dividi(10, 0));
    }
}
```

JUnit trova automaticamente tutte le classi in `src/test/java` con metodi annotati con `@Test`.

Esegue i test uno per uno. Mostra i risultati (pass/fail) nell'IDE o in console.

Ogni test usa la classe “vera” (Calcolatrice), ma non fa parte di essa.

Maven riconosce automaticamente tutti i test nella cartella `src/test/java`.



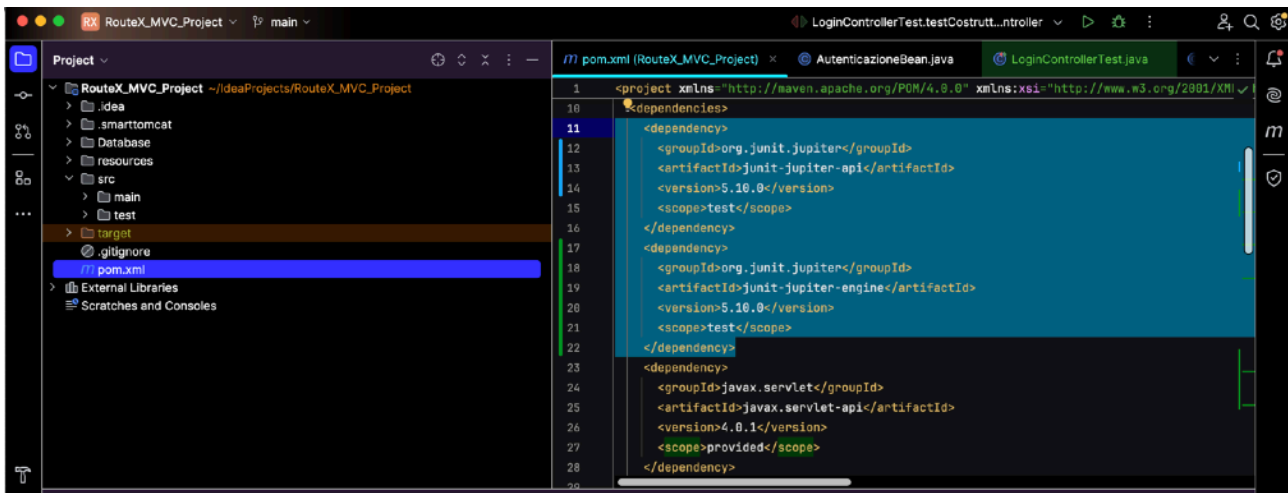
INTELLIJ E TEST - GUIDA ALLA CONFIGURAZIONE



Le dipendenze da inserire nei progetti Maven WebApp nel `pom.xml` sono le seguenti:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-engine</artifactId>
  <version>5.10.0</version>
  <scope>test</scope>
</dependency>
```



Come si può notare, oltre ad avere installato le dipendenze JUnit nel pom.xml, sullo stesso livello di main, esiste la cartella test contenente i test dell'applicativo.

La dipendenza *junit-jupiter-engine* contiene il **motore vero e proprio** che esegue i test Jupiter.

È quello che permette a Maven, IntelliJ, o Gradle di “capire” i test scritti con `@Test` e farli girare. Senza l'engine, puoi **scrivere** i test, ma non puoi **eseguirli**. È come avere una macchina senza il motore.

La dipendenza *junit-jupiter-api* contiene le classi e annotazioni che usi nel codice del test, ed è ciò che usi per scrivere i test. Questa non funziona senza il motore.

Cos'è JUnit Jupiter?

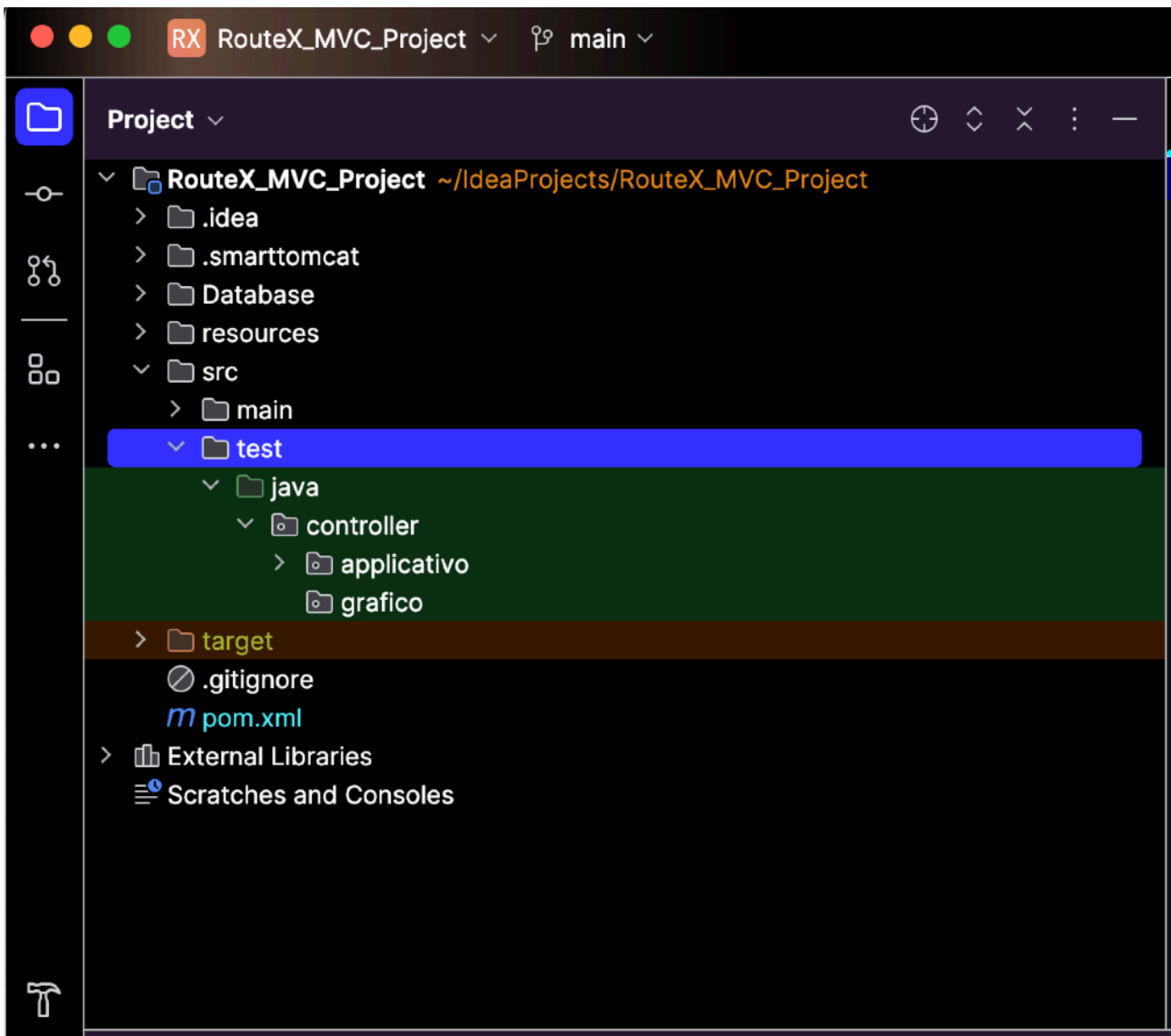


“Jupiter” è semplicemente il nome in codice del modulo principale di JUnit 5.

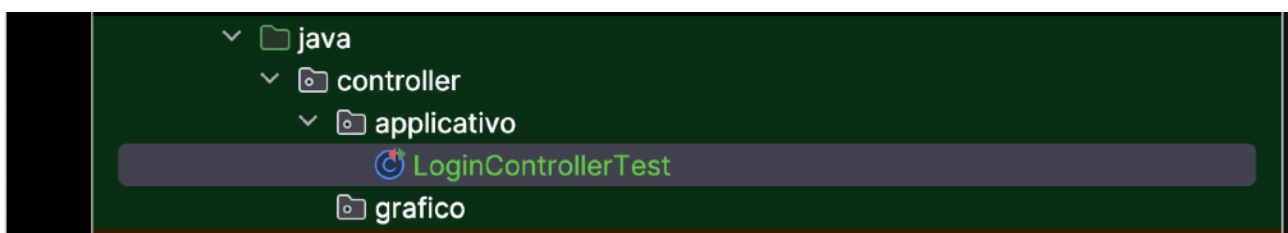
Gli sviluppatori di JUnit (tra cui Sam Brannen e Marc Philipp) scelsero **nomi planetari** per i moduli di JUnit 5 per simboleggiare **una galassia di componenti indipendenti**, invece di un unico “blocco monolitico” come JUnit 4.

A questo punto, aprendo il package dei test del mio progetto, troverò a mia volta ulteriori package.

Andiamo a vedere un test di prova e verifichiamo se passa.



Entro in controller->applicativo:



Il codice del test è il seguente:

```
package controller.applicativo;

import Bean.AutenticazioneBean;
import Controller.Applicativo.LoginController;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertNotNull;
```

```

class LoginControllerTest {

    @Test
    void testCostruttoreLoginController() {
        // Arrange: creo un bean con email e password fittizi
        AutenticazioneBean bean = new
AutenticazioneBean("test@example.com", "password123");

        // Act: creo il controller usando il bean
        LoginController controller = new
LoginController(bean);

        // Assert: verifico che l'oggetto non sia nullo
        assertNotNull(controller, "Il controller non dovrebbe
essere nullo");
    }
}

```

`assertNotNull()` è un metodo di JUnit usato nei test automatici per verificare che un oggetto non sia **null**. Se invece è **null**, il test fallisce

Sintassi

`assertNotNull(Object actual);`

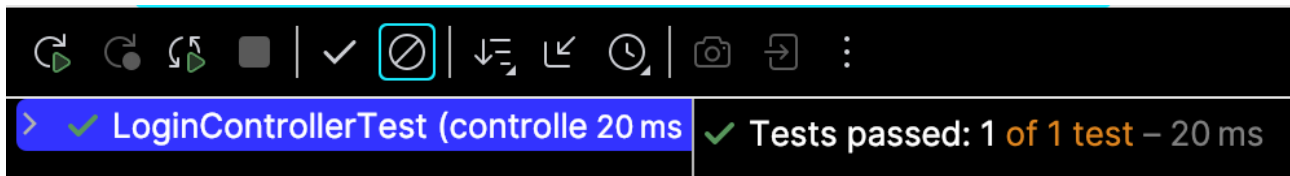
`assertNotNull(Object actual, String message);`

`actual` è l'oggetto o variabile che vuoi controllare.

`message` (opzionale) è il messaggio personalizzato che JUnit mostra se il test fallisce.

Il test che ho scritto **verifica che il costruttore di LoginController funzioni correttamente**, cioè che, dato un `AutenticazioneBean` valido (con email e password),
il costruttore crei un oggetto `LoginController` non nullo.

Eseguiamolo.



Il test è passato.



Cos'è MAVEN?

Apache Maven è uno strumento di **build automation** e **project management** per progetti Java (e non solo), sviluppato dalla Apache Software Foundation.

Il cuore di Maven è il file **pom.xml**, che contiene: le **informazioni sul progetto** (nome, versione, descrizione), le **dipendenze** da altre librerie e i **plugin** usati durante la build.

Maven scarica e aggiorna automaticamente tali librerie da repository remoti, evitando conflitti di versione e semplificando l'integrazione.

A questo punto le mie spiegazioni possono risultare superflue, poiché l'impalcatura software è ormai una scelta a discrezione.

Cos'è TOMCAT?



Apache Tomcat è un application server leggero (più precisamente, un

Servlet container) sviluppato dalla Apache Software Foundation.

Implementa le specifiche **Jakarta Servlet**, **Jakarta JSP** e **Jakarta WebSocket**, consentendo l'esecuzione di applicazioni web Java conformi a tali standard.

A questo punto, la scelta della tecnologia non rappresenta più un vincolo oggettivo, ma dipende direttamente da voi, in base alle vostre esigenze e preferenze operative. L'infrastruttura software e l'architettura di sviluppo possono quindi essere definite liberamente, adattandosi al contesto progettuale e agli strumenti che ritenete più idonei. Di conseguenza, le spiegazioni tecniche fornite finora diventano in parte superflue, poiché l'impalcatura tecnologica finale sarà determinata dalle vostre scelte. In definitiva, la tecnologia non è più un elemento sul quale posso intervenire direttamente: la sua scelta e implementazione dipendono ora interamente da voi, dalle vostre valutazioni e dal contesto in cui intendete operare.

Il mio ruolo, in questa fase, è esclusivamente quello di fornire un quadro di riferimento e di assicurare coerenza concettuale tra le vostre decisioni tecniche e gli obiettivi generali del progetto.

Fermo restando che, qualsiasi sia la vostra tecnologia, la possibilità di svolgere dei test deve essere sempre accessibile.

Il professor Falessi utilizza un ambiente di sviluppo integrato (IDE) diverso da IntelliJ, denominato **Eclipse**. Tuttavia, il suo funzionamento è sostanzialmente analogo: la gestione del progetto e la suddivisione dei package seguono la stessa logica e rispettano la medesima struttura gerarchica prevista in IntelliJ IDEA.

Esempio Falessi.

```
1 package test;
2 import static org.junit.Assert.*;
3 import org.junit.Test;
4 import logic.perfectCalculator;
5
6 public class TestBasicCalculations {
7     @Test
8     public void TestDivision(){
9         perfectCalculator TestDivision = new perfectCalculator();
10
11         assertEquals((double)2, TestDivision.PerfectDivision(22,10), 0);
12     }
13
14
15
16 }
```

Il test passerà solo se il metodo PerfectDivision(22,10,0) restituisce **esattamente 2.0** e non genera eccezioni.

Se ci fosse 1 come valore atteso, $|1.0 - 2.0| \leq 0.0$, il test fallirebbe.

E questa è la classe applicativa:

```
1 package logic;
2
3
4 public class perfectCalculator {
5
6
7
8     public int PerfectSum (int a, int b){
9         int result;
10         result = a + b;
11         return (result);
12     }
13
14     public double PerfectDivision (int a, int b){
15         double result;
16         result = a / b;
17         return (result);
18     }
19
20
21
22 }
```

Due package diversi all'interno dello stesso progetto.

Al seguente link è presente la lista ufficiale della documentazione di JUnit per le asserzioni:

<https://docs.junit.org/5.0.1/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

Puoi provare anche su:

<https://docs.junit.org/5.11.0/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html>

ANCORA COVERAGE

Il prof. Falessi distingue tre situazioni limite in cui il coverage può essere ingannevole. Il Prof lo chiama Breaking Coverage.

	Coverage	Failures	Bugs
Pessimistic	100%	> 0	0
Optimistic	100%	0	> 0
Impossible	$< 100\%$	-	-

Optimistic Coverage: 100% coverage non significa assenza di bug. È una copertura ottimistica. Illusione di aver fatto bene.

Pessimistic Coverage: Nel Pessimistic Coverage, i test di solito passano tutti, ma il tool di coverage segnala una percentuale inferiore di copertura.

Impossible Coverage: Coverage $< 100\%$ e non è possibile raggiungere 100%. In alcuni casi, non puoi fisicamente eseguire tutto il codice, anche volendo.

Il coverage è una metrica utile per capire quanto i test esplorano il codice, ma non è una misura diretta della qualità o correttezza del software.

Adesso procediamo con alcuni esempi di coverage.

Optimistic Coverage.

```
public int dividi(int a, int b) {  
    return a / b;  
}
```

@Test

```
void testDivisione() {  
    int risultato = dividi(10, 2); // OK  
    assertEquals(5, risultato);  
}
```

Coverage: 100% (tutte le righe eseguite)

Bug: se provo dividi(10, 0), eccezione ArithmeticException

Test passato: sì

Conclusione: coverage ottimo ma test poco significativo, bug nascosto.

Pessimistic Coverage.

```
public int somma(int a, int b) {  
    return a + b;  
}
```

@Test

```
void testSomma() {  
    int risultato = somma(2, 2);  
    assertEquals(5, risultato); // Errore nel test, non nel  
codice  
}
```

Coverage: 100%

Failure: sì (test fallisce)

Bug nel codice: no, è corretto

Conclusione: i test ti fanno credere che ci sia un problema, ma in realtà è il test ad essere sbagliato.

Impossible Coverage.

```
public void salvaSuFile(String path, String contenuto) {  
    try {  
        FileWriter writer = new FileWriter(path);  
        writer.write(contenuto);  
        writer.close();  
    } catch (IOException e) {  
        System.err.println("Errore di scrittura su disco  
pieno");  
    }  
}
```

```

}

@Test
void testScritturaFile() {
    salvaSuFile("test.txt", "ciao");
    // Come posso simulare un "disco pieno"?
}

```

Coverage: <100%

Non puoi eseguire il blocco catch realisticamente.

Conclusione: anche con test perfetti, alcune parti del codice non saranno mai raggiunte.

Non sempre esiste un modo realistico per scatenare l'errore.

La parte di codice dentro il **catch** ("Errore di scrittura su disco pieno")
viene eseguita **solo** se il disco è davvero pieno.

Non puoi svuotare o riempire un disco apposta per un test automatizzato.

Non puoi costringere la **FileWriter** a fallire a comando.

Quindi, non puoi ottenere il 100% di coverage reale, anche se scrivi **assertThrows**.

Nota: `assertThrows(ExpectedException.class, executable)`,

Serve per verificare che un metodo lancia un'eccezione specifica.

```

@Test
void testDivisionePerZero() {
    assertThrows(ArithmeticException.class, () -> {
        int risultato = 10 / 0;
    });
}

```

Qui il test passa perché il codice genera effettivamente un'ArithmeticException.

Nel progetto c'è una sezione dedicata al testing.

4 Testing

- Develop at least 3 test cases per person. In each test (class) file, please report (via Java comments) the name of the person in charge.

Significa:

1. Ogni membro del gruppo deve scrivere almeno 3 test case (cioè 3 metodi di test, ciascuno annotato con **@Test**).

2. Ogni file Java che contiene i test (quindi ogni **classe di test**, come LoginControllerTest, PagamentoDAOTest, ecc.) deve contenere **un commento** con il **nome dello studente** che li ha realizzati.

```
/**  
 * Author: Simone Remoli  
 * Class under test: LoginController  
 * Description: Unit tests for login logic  
 */
```

Codice colori nella pratica.

Significato visivo (i colori nel codice)

-  **Rosso:** codice mai eseguito da un test
-  **Verde:** codice eseguito almeno una volta
-  **Giallo:** ramo parzialmente coperto (es. if eseguito solo in una condizione)

Ma se una riga non ha colore né testo accanto? Non è una riga di codice eseguibile, quindi IntelliJ non la considera nel coverage.

Inoltre, nella relazione è presente il seguente punto:

Exceptions: at least 2 per member (do not just catch and back-propagate the exceptions, but properly handle them. Possibly define your own error logic by means of exceptions)

Eccezioni: almeno due per membro del gruppo.

Non limitatevi a catturare e rilanciare le eccezioni (back-propagate), ma gestitele correttamente. Se possibile, **definite una vostra logica di errore personalizzata** tramite eccezioni proprie.

Mi sembra tutto abbastanza chiaro e preciso. Inequivocabile. **Devi mostrare che il programma sa reagire a un'eccezione: chiudendo risorse, notificando l'utente, scrivendo nel log o traducendo l'errore in un tipo più leggibile.**

Quindi, ricapitolando:

Nel *testing coverage*, l'**Optimistic Coverage** si verifica quando **tutti i test sembrano andare bene (0 errori)** e il sistema raggiunge un **coverage del 100%**, ma in realtà il **codice non è completamente corretto**. In altre parole: **il coverage è ottimista**, perché dà un'illusione di completezza, ma **non tiene conto di situazioni particolari o input non testati** che potrebbero generare errori.

È necessario fornire esempi significativi da presentare in sede d'esame, accompagnati da una breve descrizione del problema e dal relativo codice. La copertura dei test deve risultare pari al 100%.

Esempio:

Nell'esempio la classe `StupidStringCompare` contiene un metodo che effettua la comparazione di stringhe prendendo un carattere alla volta di entrambe le stringhe in input e facendo un confronto tra esse. L'algoritmo utilizzato funziona bene nei casi in cui le stringhe passate in input al metodo hanno la stessa lunghezza e nel caso in cui la prima stringa sia più corta della seconda, ma lancia l'eccezione `IndexOutOfBoundsException` quando invece è la seconda stringa ad avere una lunghezza maggiore, dal momento che durante la comparazione il sistema prova ad accedere a caratteri oltre la lunghezza della stringa stessa. Tale eccezione non viene evitata a priori effettuando un controllo sugli input, e pertanto può essere generata durante l'utilizzo dell'applicazione da parte dell'utente.

Si tratta di coverage ottimistico, dal momento che entrambi i test passano con 0 errori e 0 failures e ho il 100% di coverage, nonostante esistano particolari combinazioni degli input che possono generare bug nel sistema.

Hai una classe **StupidStringCompare** che confronta due stringhe carattere per carattere. Funziona solo se le due stringhe hanno la stessa lunghezza.

Se una stringa è più lunga dell'altra, il metodo tenta di accedere a un indice oltre la lunghezza della stringa più corta, e quindi genera una `IndexOutOfBoundsException`.

Durante i **test automatici**, si sono usati solo casi in cui le due stringhe hanno la stessa lunghezza.

Quindi:

- I test **non falliscono** (nessuna eccezione).
- Il coverage risulta **100%** (tutti i rami eseguiti).
- Ma **non si è testato un caso reale** che può generare errore (stringhe di lunghezza diversa).

Il sistema di test è “**ottimista**”, perché considera il codice completamente corretto, **ma in realtà non lo è**: ci sono casi non coperti che generano bug.

Quindi, Il metodo confronta due stringhe carattere per carattere, ma non verifica che abbiano la stessa lunghezza. I test usano solo stringhe della stessa lunghezza, quindi tutti i test passano e la copertura risulta del 100%, ma un caso reale può generare un'eccezione (`IndexOutOfBoundsException`).

```
public class StupidStringCompare {
    public static boolean compare(String s1, String s2) {
        for (int i = 0; i < s1.length(); i++) {
            if (s1.charAt(i) != s2.charAt(i))
                return false;
        }
        return true;
    }
}
```

E i test sono:

```
import org.junit.Test;
import static org.junit.Assert.*;

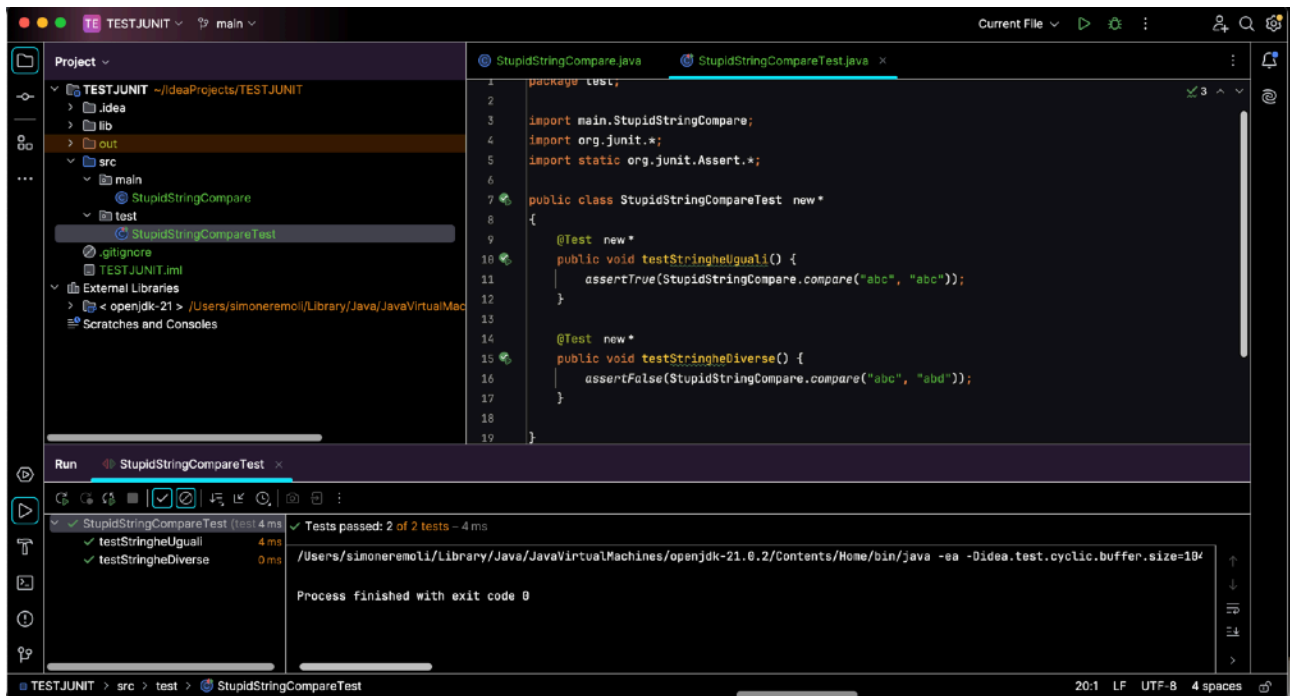
public class StupidStringCompareTest {

}
```

Tutte le righe di codice sono eseguite, **coverage 100%**. Ma `compare("abc", "abcd")` genera `StringIndexOutOfBoundsException`, quindi **Optimistic Coverage**.

Questo è l'esempio sul mio IntelliJ, vedete anche la struttura delle cartelle.

`assertFalse(condizione)`: Il test **passa** se l'espressione è **falsa**, e **fallisce** se l'espressione è **vera**.



assertTrue(condizione): Il test passa se l'espressione è vera, e fallisce se l'espressione è falsa.

1. Test: `assertTrue(StupidStringCompare.compare("abc", "abc"))`;

Le due stringhe sono identiche. Il ciclo confronta: `a==a`, `b==b`, `c==c` e quindi nessuna differenza, ritorna true. Il test **Passa**.

2. Test: `assertFalse(StupidStringCompare.compare("abc", "abd"))`;

Le stringhe differiscono all'ultimo carattere (`c != d`). Il ciclo trova la differenza e ritorna false. Il test **Passa**.

Adesso supponiamo che io avessi inserito un nuovo test:

```
@Test
public void testStringheLunghezzaDiversa() {
    assertFalse(StupidStringCompare.compare("abcdef",
"abd"));
}
```

Le stringhe hanno **lunghezze diverse**. Il ciclo usa `s1.length() = 6`, quindi tenterà di leggere:

- `s1.charAt(0) → ok`
- `s1.charAt(1) → ok`
- `s1.charAt(2) → ok`
- ma quando `i = 3`, prova `s2.charAt(3)` e giustamente c'è n'errore.

Risultato: `java.lang.StringIndexOutOfBoundsException`. L'ultimo valore è `false`, prima di `Exception`.

Dove si deve gestire l'eccezione? la gestione va fatta nella parte logica, cioè nel metodo da testare, non nel test. Se il metodo lancia eccezioni non gestite, significa che **non è robusto**.

Ma se il metodo va in errore, il test **non sa** cosa significhi quell'errore, per lui l'eccezione non è un fallimento *logico* (a meno che tu non glielo dica esplicitamente). Ecco perché **“passa”**: il test si limita a vedere se l'assert non fallisce, non se il metodo è scritto in modo robusto.

Se aggiungo il seguente test:

```
@Test(expected = StringIndexOutOfBoundsException.class)

public void testEccezione() {
    StupidStringCompare.compare("abcdef", "abd");
}
```

Il test ovviamente non passa, perché mi aspetto che venga generata l'eccezione, e fatalità si verifica.

Se scrivo il seguente test, non passerà mai, perché l'ultimo valore è `true`, prima di `exception`:

```
public void testStringheLunghezzaDiversa() {
    assertFalse(StupidStringCompare.compare("abcdef",
"ab"));
}
```

```
19      @Test new *
20      public void testStringheLunghezzaDiversa() {
21          assertFalse(StupidStringCompare.compare("abcdef", "ab"));
22      }
```

Il test è fallito infatti, ultimo valore uguale a true.

```
19      @Test new *
20      public void testStringheLunghezzaDiversa() {
21          assertFalse(StupidStringCompare.compare("abcdef", "abd"))
22      }
```

Il test è passato, ultimo valore uguale a false, prima di exception.

ESEMPIO 2.

```
package main;
```

```
public class StupidStringCompare {

    public static boolean compare(String s1, String s2) {

        // Controllo preliminare: se una stringa è nulla:
        // confronto non valido
        if (s1 == null || s2 == null) {
            System.out.println("Una delle stringhe è
nulla.");
            return false;
        }

        // Controllo sulla lunghezza
        if (s1.length() != s2.length()) {
            System.out.println("Le stringhe hanno lunghezze
diverse.");
            return false;
        }

        // Confronto carattere per carattere
        for (int i = 0; i < s1.length(); i++) {
            if (s1.charAt(i) != s2.charAt(i)) {
                System.out.println("Differenza trovata al
carattere " + i);
                return false;
            }
        }
    }
}
```

```

    }

    // Tutti i caratteri uguali
    return true;
}
}

```

I TEST:

```

package test;

import main.StupidStringCompare;
import org.junit.*;
import static org.junit.Assert.*;

public class StupidStringCompareTest {

    @Test
    public void testStringheUguali() {
        assertTrue(StupidStringCompare.compare("abc",
"abc"));
    }

    @Test
    public void testStringheDiverse() {
        assertFalse(StupidStringCompare.compare("abc",
"abd"));
    }

    @Test
    public void testStringheLunghezzaDiversa() {
        assertFalse(StupidStringCompare.compare("abcdef",
"abd"));
    }

    @Test
    public void testStringaNulla() {
        assertFalse(StupidStringCompare.compare(null,
"abc"));
    }
}

```

Tutti i test passano in questo caso.